

Estimated time needed: 110 minutes

Job Recommendation System Introduction

In this lab, you will design a job recommendation system using embeddings and similarity search. You will use Hugging Face's HfInference for generating text embeddings and a database that stores vectors for storing and querying these embeddings. The system will recommend job postings based on a given query, such as finding positions for a specific job title in a particular location.

In part 2, you will allow candidates to provide their resume in a PDF format to this **Job Recommendation System**. It will also recommend jobs from the database based on the skill set and experience by extracting information from the resume and looking for matches in the data set to recommend jobs.

You will turn in your code for Part 1 and Part 2 as well as provide screenshots of your output.

Learning objectives

After completing this lab, you will be able to:

- Develop a recommendation system using a database that stores vector data and *Hugging Face* tools for generating vector embeddings
- Use pretrained NLP models for classification and similarity searches to extract and match criteria from natural language queries.
- Accurately return items in the database that match users' search terms via recommendation algorithms
- Refine and optimize recommendation accuracy for user searches
- Allow users to search the database by providing a PDF

Prerequisites

- Basic knowledge of Hugging Face tools
- Intermediate knowledge of Node.js
- Intermediate knowledge of databases which store vector data

Important notice about this lab environment

Skills Network Cloud IDE (based on Theia and Docker) is an open-source IDE (Integrated Development Environment) that provides an environment for hands-on labs in course and project-related labs.

Please be aware that sessions for this lab environment are not persistent. Every time you connect to this lab, a new environment is created for you.

You will lose data if you exit the environment without saving to GitHub or another external source.

Plan to complete these labs in a single session to avoid losing your data.

Part 1 Task 1: Setting up the environment and populating the database

For the final project, you may choose which vector database you would like to use. The practice project used ChromaDB. Where appropriate in this lab we will provide hints for *Cassandra*, *MongoDB*, and *PostgreSQL*.

Information on working with NoSQL and SQL databases and vector data can be found in courses 2 and 3 respectively:

- Vector Search with NoSQL Databases using MongoDB & Cassandra
- Vector Search with Relational Databases using PostgreSQL

Important: You will turn in your code for Part 1 and Part 2 as well as provide screenshots of your output. Be sure to save your code in a file outside of the IDE environment so you can submit it.

Part 1 Task 1: A. Run Docker

1. In your IDE environment, select the **Terminal** tab at the top-right of the window and then select **New Terminal** from the drop-down menu.
2. Run the Docker command to pull an image.

Hint: Use docker run command

- ▶ Sample solution for Chroma DB
- ▶ Sample solution for MongoDB
- ▶ Sample solution for Cassandra
- ▶ Sample solution for PostgreSQL

Important! Please keep this terminal window open. The lab requires Docker execution until the end of the lab.

Part 1 Task 1: B. Install the database environment

1. Install the database environment using a new terminal window and press **Enter**.

Note: You may create a lab environment using *ChromaDB*, *MongoDB*, *Cassandra*, or *PostgreSQL* by importing the necessary packages for these databases.

- For Chroma DB, you need to install chromadb.
- For mongoDb, you need to install mongodb.
- For cassandra, you need to install cassandra-driver vectodb.
- For PostgreSQL, you need to install pg pgvector.

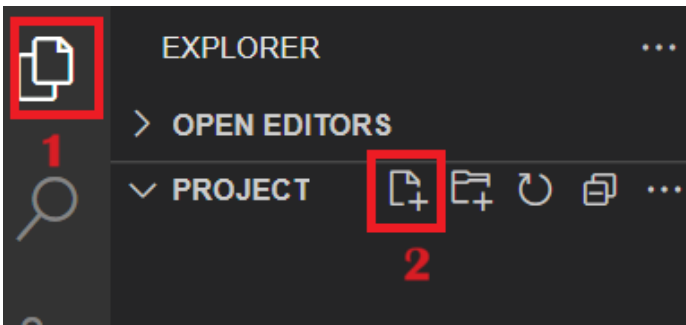
2. Install one more dependency by performing the following command in the same terminal window where you installed the previous dependency.

```
npm install @huggingface/inference
```

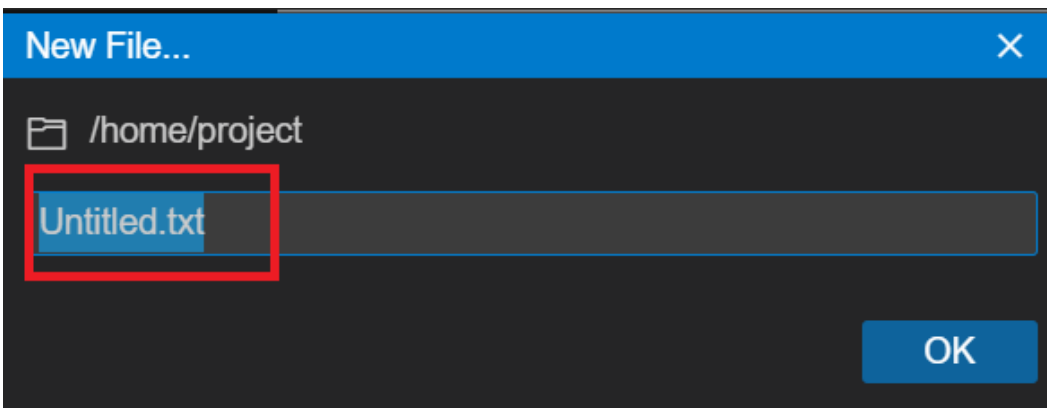
The @huggingface/inference package provides capabilities to use Hugging Face inference to use AI-based models.

Note: The free version of the Hugging Face API key only allows a limited number of accesses in a given time period, although the documentation doesn't make that limit clear. It does seem like the limit is on an hourly basis, so if you get an error after frequent accesses to their API, you may need to wait up to an hour until you can continue your work.

3. Select **Explorer** on the left side of the terminal window, as shown at number 1 in the following screenshot. Then, within the **Project** folder, select **New File** as shown at number 2 in the following screenshot.



4. A pop-up box should display with the default file name **Untitled.txt**. Change the default name to `jobRecommendationSystem.js`.



Part 1 Task 1: C. Import necessary modules

1. Import the appropriate class into your `jobRecommendationSystem.js` file to create an instance of the client that interacts with your preferred vector database.
 - ▶ See sample solution for Chroma DB
 - ▶ See sample solution for MongoDB
 - ▶ See sample solution for Cassandra
 - ▶ See sample solution for PostgreSQL
2. Now import the Hugging Face inference model and include your Hugging Face API key. Name the new instance of the model `hf`.

Hint: To import the class, use the `require` keyword.

Part 1 Task 1: D. Get the data

1. First, you need to obtain a copy of the data. Execute the following command to obtain the data from the `jobPostings.js` file under the **Project** folder.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/PuG2Nqinq_7G71t1NP0JeQ/jobPostings.js
```

2. This data set includes multiple objects that holds keys. Each key provides specific information about a job:

- jobId
- jobTitle
- company
- location
- jobType
- salary
- jobDescription
- jobResponsibilities
- preferredQualifications
- applicationDeadline

3. Require the data in `jobRecommendationSystem.js`.

Part 1 Task 1: E. Create the collection or table

In this section, you will create a collection out of the provided data set, `jobPostings.js` within the `main()` function.

1. Create a variable named `collectionName` and assign the string `job_collection` to it in `jobRecommendationSystem.js`.
2. Create the asynchronous `main()` function in `jobRecommendationSystem.js` file with a try-catch block. This function will contain the code for interacting with the database.
3. Now create the collection (if you're using Chroma DB or MongoDB) or table (if you're using Postgres or Cassandra) inside the try block of the `main()` function.

- ▶ Sample solution for Chroma DB
- ▶ Sample solution for Mongo DB
- ▶ Sample solution for Cassandra
- ▶ Sample solution for the PostgreSQL

Final project deliverable

You will turn in your `jobRecommendationSystem.js` file for your final project. You will be graded on these elements:

- the models and data you require
- the variables you declared associated with these require statements

Part 1 Task 2: Create IDs and Generate Embeddings

Reminder: Do not close the terminal where Docker is running.

Part 1 Task 2: A. Create unique IDs

Now after creating the collection (or table) in the `main()` function, you need to create unique job IDs using the `Set()` method for all of the job data objects required from the variable named `jobPostings`.

Use these guidelines:

- Use the `Set()` method to define a set named `uniqueIds` that will store the unique job IDs.
- Use a `forEach()` method to iterate over each job object and its index in an array named `jobPostings`.
- Use a `while` loop to check if the job ID already exists in the `uniqueIds` set.
- If the job ID exists in the set, the function should append the index to the job ID to make it unique.

Note: For MongoDB, Cassandra, and PostgreSQL, you do not have to create unique IDs separately.

2. Process an array of `jobPostings` to generate embeddings and store the data in a collection within the `main()` function.

- Create text descriptions
 - Transform each job into a string combining its `jobTitle`, `jobDescription`, `jobType`, and `location`.
 - Create a variable named `jobTexts` and store the resulting array of text descriptions in it.
- Generate embeddings
 - Call a `generateEmbeddings()` function (which you will define later), with `jobTexts` as a parameter to create embeddings for the text descriptions.
 - Store the result in a variable named `embeddingsData`.
- Store the data in the collection or table

- ▶ Sample solution for Chroma DB
- ▶ Sample solution for MongoDB
- ▶ Sample solution for Cassandra
- ▶ Sample solution for PostgreSQL

Final project deliverable

You will turn in your `jobRecommendationSystem.js` file for your final project. You will be graded on these elements:

- You have an assigned a variable named `uniqueIds` to a new `Set()` object.
- The method you wrote that follows the declaration of `uniqueIds` that
 - uses a `forEach()` method to iterate over each job object in the new `Set`
 - uses a `while` loop to check if the job ID already exists in the `uniqueIds` set

Part 1 Task 2: B. Create function to generate embeddings

Now, let's write the function to actually generate the embeddings. It will use a function from the Hugging Face model named `featureExtraction()`.

1. Create a function named `generateEmbeddings()` outside of the `main()` function. This function generates embeddings for the user's search strings.
2. For this, you will need to call the `featureExtraction()` method from the Hugging Face model in your `generateEmbeddings()` function. Use this function to specify the model you want to use for feature extraction.
 - Use the `model` for feature extraction, in this case, "sentence-transformers/all-MiniLM-L6-v2".
 - Your function should wait for the `hf.featureExtraction()` call to complete using `await` and store the resulting embeddings in a variable named `results`.
 - Lastly, your function should return the `results`.

Final project deliverable

You will turn in your `jobRecommendationSystem.js` file for your final project. You will be graded on:

- Your `generateEmbeddings()` function

Part 1 Task 3: Extract criteria to filter the data

For this task, you will use the queries to extract criteria to filter your data. You will write three functions in this section.

Part 1 Task 3: A. Create the function to classify text and add labels

You will define a function that will classify input text according to one or more provided candidate labels. It will use the BART large model, fine-tuned for multi-label natural language inference (MNLI). In part, the function will return an object with a score property between 0 and 1, indicating how close the model finds the label to the input text. You can use this function for tasks such as sentiment analysis, topic classification, and other text classification problems.

1. First, write a helper function to classify text and labels named `classifyText()`. This function classifies provided text into one of four categories: *location*, *job title*, *company*, or *job type*.
- Define an asynchronous function named `classifyText()` that takes two parameters: `word` and `labels`.
 - In the function's body, send a `request()` to the Hugging Face API using `hf.request()`, assuming that when you required the Hugging Face model, you named the variable `hf`.
 - Use the "facebook/bart-large-mnli" model for natural language inference (NLI).
 - Provide text as input to the model.
 - The function returns a response object from the API.

Final project deliverable

You will turn in your `jobRecommendationSystem.js` file for your final project. You will be graded on your `classifyText()` function.

Part 1 Task 3: B. Create a function to extract the criteria

Write a function named `extractFilterCriteria()` outside of the `main()` function, which allows you to extract criteria from the data and classify it according to a given set of labels. It will use the `classifyText()` helper function.

This asynchronous function will extract the location, job title, job type, and company name from a query string. Follow the steps below to create `extractFilterCriteria()`.

1. The function should take a single parameter, `query`.
2. Initialize an object named `criteria` with the `location`, `jobTitle`, `jobType`, and `company` properties, set to `null`.
3. Then create one variable named `labels` which is initialized as array with elements such as `location`, `job title`, `company`, and `job type`.
4. Define the variable named `words` and assign `query` to `words` using the `split()` method.
5. Call the `classifyText()` function with `word` and `labels` as parameters and assign the result to a variable named `result`.
6. The `result` contains `labels` and `scores`.
7. Store the `label` with the highest score in `highestJobScoreLabel`, and its highest score in `score` variable.
8. The code checks if a score is greater than 0.5 and, based on the value of `highestScoreLabel`, updates the `criteria` object with the word under properties like `location`, `jobTitle`, `company`, or `jobType`. If `highestScoreLabel` doesn't match any case, it does nothing.
9. Write a conditional statement that checks if the classification score is greater than 0.5. If the score exceeds this threshold, it means the classification is considered confident enough to proceed.
 1. `switch (highestScoreLabel)`: This switch statement determines what action to take based on the label with the highest classification score.
 2. `case location`: If the highest score label is "location," it sets the `criteria.location` to `word`.
 3. `case job title`: If the highest score label is "job title," it sets the `criteria.jobTitle` to `word`.
 4. `case company`: If the highest score label is "company," it sets the `criteria.company` to `word`.
 5. `case job type`: If the highest score label is "job type," it sets the `criteria.jobType` to `word`.
 6. Returns the `criteria` object.

► Sample solution for `extractFilterCriteria()`

Note: You can modify the code in the sample solution according to your chosen database such as **MongoDB**, **Cassandra**, or **PostgreSQL**.

- Include a variable named `query` inside of the `main()` function before the `try` block in the `jobRecommendationSystem.js` file and assign it the string value "Creative Studio".

- Within the try block of the `main()` function, call the `extractFilterCriteria()` function with `query` as a parameter. Assign the return value to a variable named `filterCriteria`.

Part 1 Task 4: Create a function to perform the similarity search

Create the `performSimilaritySearch()` Function and create collection

Next, you need to perform a similarity search to compare your user query embeddings to your collection data embeddings.

1. Create an asynchronous function named `performSimilaritySearch()` outside of the `main()` function. It should calculate the cosine similarity using embeddings and three arguments: `collection`, `queryTerm`, and `jobPostings`. It should compare the embedding from the `queryTerm` and the job posts in the `collection`.
2. Generate the embeddings by making a call to the `generateEmbeddings()` method for the `queryTerm` and assign the value to a variable named `queryEmbedding` and request up to three results.

- Sample solution for Chroma DB
- Sample solution for MongoDB
- Sample solution for Cassandra
- Sample stolution for PostgreSQL

3. Handle the cases if no similar results were found or `results.length === 0` by logging a message and returning an empty array.
4. Map the top matching food items by creating a variable named `topJobPostings`. Assign it the details if `results` are found by mapping over the `ids` array in `results` to retrieve corresponding food items from the `jobPosting` array. Use their `id`, then return the matching `jobPosting`, including its similarity score from `results.distances`.
5. Sort `topJobPostings` based on their scores in ascending order `a.score - b.score`. This step ensures that the most similar items to the query appear first in the returned array.
6. Catch any errors that occur during the process, such as network errors and API failures, using the try-catch block. If an error occurs, log the error message to the console and return an empty array.
7. Within the `main()` function, assign the result of the `performSimilaritySearch()` function to a variable named `initialResults`.
8. Iterate over the first three items in the `initialResults` array inside the `main()` function and log each item's `jobTitle`, `jobType`, `jobDescription`, and `company`.

Final project deliverable

You will turn in your `jobRecommendationSystem.js` file for your final project. You will be graded on these elements from this task:

- A call to `generateEmbeddings()` with `jobTexts` as a parameter.
- A call to `performSimilaritySearch()`.
- A call to `filterJobPostings()` with two parameters, `jobPostings`, `filterCriteria`.
- A variable named `query` set to the string "Creative Studio".
- The try-catch block, where the catch logs an error to the console.

Part 1 Task 5: Check the Output

1. Open the new terminal if it is not opened. Check the output using the following command in the terminal:

```
node jobRecommendationSystem.js
```

2. Use the query `const query = "Creative Studio";`

Final project deliverable

Take a screenshot that shows your output. You will need to submit this as part of your final project.

To take a screenshot on Windows, use the snipping tool by going to the start menu and opening the snipping tool. Click “New” then click and drag over the area of the screen you want to screenshot. Press **Ctrl + S** to save the file as a .png file or a .jpg file.

To take a screenshot on MacOSX press shift+command+4 then click and drag over the area of the screen you want to screenshot. The file will be saved to your desktop.

Part 2: PDF Resume Job Recommendation System

Congratulations on completing Part 1 of the final project! You are now ready for Part 2.

In Part 2, you will create the job recommendation system similar to the one in Part 1 but you will allow the use of a PDF file, such as a resume, to find jobs with similar skills.

Part 2 Task 1: Setting up the Environment

1. Create a new file under the **Projects** folder and name it `smartRecommendationSystem.js`.
2. Open a new terminal and install the appropriate packages.

```
npm i pdf-parse @huggingface/inference chromadb
```

You need to run Docker if it is not already running. **If you have not closed out of the environment after completing Part 1 and still have Docker running, you may skip the rest of this task and continue.** Otherwise, revisit:

- Part 1 Task 1: A. Run Docker
- Part 1 Task 1: B. Install the database environment
- Part 1 Task 1: D. Get the data

Part 2 Task 2: Import and Initialize

1. Initialize and require the modules and packages in the `smartRecommendationSystem.pdf` file including:

- `fs`
- `pdf-parse`
- `@huggingface/inference`
- `readline`
- your chosen database

2. Create an instance of the Hugging Face inference client with your Hugging Face API key and assign it to a variable named `hf`.

3. Initialize your database and then assign a collection name to it by declaring a variable named `collectionName`.

4. Now require the array content of the `jobPostings.js` file into the `smartRecommendationSystem.pdf` file.

5. Execute the following command to obtain the `testResume.pdf` file that you will need for testing.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/4V1kcXr8NDjTN6Sof83-4w/testResume.pdf
```

Final project deliverable

You will turn in your `smartRecommendationSystem.js` file for your final project. You will be graded on these elements:

- The models and data you require.
- The variables you declared associated with these require statements.

Part 2 Task 3: Extract Text and Generate Embeddings

Part 2 Task 3: A. Extract text from the PDF

Create a function named `extractTextFromPDF()` in your `smartRecommendationSystem.js` file that will extract its content into a text format.

Part 2 Task 3: B. Generate embeddings

Define a function named `generateEmbeddings()` to generate the embeddings from the extracted text using the Hugging Face inference model.

Final project deliverable

You will turn in your `smartRecommendationSystem.js` file for your final project. You will be graded on these elements:

- Your `extractTextFromPDF()` function.
- Your `generateEmbeddings()` function.

Part 2 Task 4: Get the Resume

In this task, you will prompt the user for the path to the PDF and then extract the details from that resume into a buffer.

Part 2 Task 4: A. Prompt for user input

Create a function named `promptUserInput()` that prompts the user for the file path and name of the PDF. Assume that they are using a command-line interface (CLI) environment.

Part 2 Task 4: B. Extract from PDF

Now, create a function named `extractFromPDF()` outside of the `main()` function, which will extract the text data from a PDF resume file asynchronously.

Final project deliverable

You will turn in your `smartRecommendationSystem.js` file for your final project. You will be graded on these elements:

- Your `promptUserInput()` function.
- Your `extractFromPDF()`.

Part 2 Task 5: Store Embeddings

Now, create a function named `storeEmbeddings()` outside of the `main()` function. Use this function to store the embeddings from the job posts into your database.

Follow these steps to create it:

1. Initialize an empty array to hold the embeddings for the food items.
2. Create an array of empty metadata objects, one for each job post that ensures a consistent data structure and compatibility with the vector database you are using for this project.
3. Iterate over the job postings array asynchronously. Make sure you change all of the text to lowercase and store the embeddings in an array.
4. Create an array of unique string IDs based on the index of each job post.
5. Map the job posting to store the job name of each item in the variable.
6. Within the try block, create a collection and store the collection in a variable.
7. Add the IDs, documents, embeddings, and metadata to the collection.
8. Log any errors using the catch block.

► Sample Solution using Chroma DB

Part 2 Task 6: Create the Main Function

Now create a `main()` function for your workflow. It should handle these processes:

- Extracting job items from a PDF
- Generating embeddings for these items
- Storing the embeddings in the database
- Querying the database to provide job recommendations based on resume

Follow the following steps to create the `main()` function.

1. Wrap the entire process in a try-catch block to handle any errors that might occur and log them to the console.
2. Call the `storeEmbeddings()` function to store the job post embeddings and wait for it to complete.
3. Prompt the user to enter the path to their resume PDF file and wait for the user input by calling the `promptUserInput()` function with the variable `filePath` as a parameter.
4. Create a variable named `text` and call the `extractTextFromPDF()` function by passing in the `filePath`.
5. Create a variable named `resumeEmbedding`. Assign the result of the call from the function named `generateEmbeddings()` with `text` as its parameter to `resumeEmbedding`.
6. Retrieve the collection.
7. Create a variable named `results` and query the collection by specifying `resumeEmbedding` as the query embedding. Request the top 5 similar items of the similarity search.
8. Check if there are any IDs returned in the `results` and make sure the first ID in the array is not empty.
9. If the `results` array is not empty, iterate through the IDs of the results and log the job title of each recommended job from the `jobPostings` based on the retrieved IDs.
10. Log any errors encountered during the process to the console and rethrow the error.
11. Outside `main()` call `main()`.

Final project deliverable

You will turn in your `smartRecommendationSystem.js` file for your final project. You will be graded on these elements in your `main()` function:

- A call to `promptUserInput()`.
- A call to `storeEmbedding()`.
- A call to `extractTextFromPDF()`.
- A call to `generateEmbeddings()` with one parameter.
- The try-catch block, where the catch logs an error to the console.

Part 2 Task 7: Check the Output

1. Check your output using this command:

```
node smartRecommendationSystem.js
```

2. Enter the name of the PDF file you obtained at the beginning of Part 2 when prompted.

```
Enter the path to the candidate's resume PDF: dummy.pdf
```

Final project deliverable

Take a screenshot that shows your output. You will need to submit this as part of your final project.

To take a screenshot on Windows, use the snipping tool by going to the start menu and opening the snipping tool. Click "New" then click and drag over the area of the screen you want to screenshot. Press **Ctrl** + **S** to save the file as a .png file or a .jpg file.

To take a screenshot on MacOSX press shift+command+4 then click and drag over the area of the screen you want to screenshot. The file will be saved to your desktop.

Congratulations!

You successfully completed this final project. Be sure to turn in all of the required code and screenshots as part of your final project submission.

Author(s)

Richa Arora
Bethany Hudnutt

© IBM Corporation. All rights reserved.